

Building a Medical-Document RAG Project

A beginner-friendly guide to APIs, document extraction and retrieval-augmented generation

Purpose

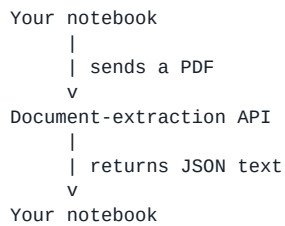
This guide explains how to extract text from medical documents, prepare the text for retrieval and generate answers grounded in trusted sources.

Medical safety note

This is an educational prototype. It must not diagnose, prescribe, replace a clinician or act as an emergency service. Use trusted sources, keep citations, avoid identifiable patient information and direct personal medical questions to a qualified healthcare professional.

1. What is an API?

An API allows one program to communicate with another. In this project, a notebook sends a document to an extraction API, and the API returns extracted text as JSON.



Example response:

```
{
  "filename": "guideline.pdf",
  "text": "The extracted document text..."
}
```

Two approaches are possible:

- Use a third-party extraction API with its URL, API key and request format.
- Build your own API, which receives a PDF and returns extracted text.

2. Build a simple document-extraction API

This example uses FastAPI and pypdf. It works best with PDFs that contain selectable text.

```
pip install fastapi uvicorn python-multipart pypdf
```

Create a file named `extract_api.py`:

```
from io import BytesIO
from fastapi import FastAPI, UploadFile, File, HTTPException
from pypdf import PdfReader

app = FastAPI()

@app.post("/extract")
async def extract_document(file: UploadFile = File(...)):
    if not file.filename.lower().endswith(".pdf"):
        raise HTTPException(status_code=400,
                            detail="Only PDF files are supported.")

    file_bytes = await file.read()
    reader = PdfReader(BytesIO(file_bytes))
    pages = []

    for page_number, page in enumerate(reader.pages, start=1):
        text = page.extract_text() or ""
        pages.append({"page": page_number, "text": text})

    full_text = "\n\n".join(page["text"] for page in pages)
    return {"filename": file.filename,
            "page_count": len(pages),
            "text": full_text,
            "pages": pages}
```

Start the API from the terminal:

```
uvicorn extract_api:app --reload --port 8000
```

The endpoint is available at **<http://127.0.0.1:8000/extract>**.

3. Call the API from a notebook

The notebook sends a PDF using an HTTP POST request. The API response is converted from JSON into a Python dictionary.

```
import requests

pdf_path = "medical_guideline.pdf"
with open(pdf_path, "rb") as pdf_file:
    response = requests.post(
        "http://127.0.0.1:8000/extract",
        files={"file": ("medical_guideline.pdf", pdf_file, "application/pdf")}
    )

response.raise_for_status()
result = response.json()
print(result["filename"])
print(result["page_count"])
print(result["text"][:1000])
```

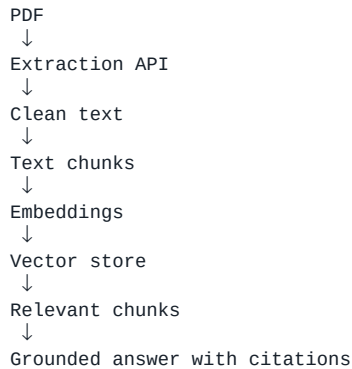
The important steps are:

- `requests.post(...)` sends the PDF to the API.
- `response.json()` reads the JSON response.
- `result["text"]` retrieves the extracted text.

If a scanned PDF returns empty text, OCR is required. Start with a text-based PDF so the API workflow is easy to verify.

4. Connect extraction to RAG

After extraction, the text is split into chunks. The chunks are embedded, stored and searched when a question is asked.



Simple chunking example:

```
document_text = result["text"]
chunks = [document_text[i:i + 1000] for i in range(0, len(document_text), 850)]
print(f"Created {len(chunks)} chunks")
```

Attach metadata to each chunk, such as document title, publisher, publication date, page number and URL. This allows the final answer to cite its sources.

5. Add embeddings and retrieval

An embedding converts text into numbers that represent meaning. The question and document chunks are embedded in the same way, then compared to retrieve the closest chunks.

Install the client and numerical library:

```
pip install openai numpy python-dotenv
```

Create a .env file and keep the key private:

```
OPENAI_API_KEY=your_api_key_here
```

Create embeddings:

```
import os
import numpy as np
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
EMBEDDING_MODEL = "text-embedding-3-small"

def embed_texts(texts):
    response = client.embeddings.create(model=EMBEDDING_MODEL, input=texts)
    return np.array([item.embedding for item in response.data], dtype=np.float32)

def normalise(vectors):
    norms = np.linalg.norm(vectors, axis=1, keepdims=True)
    return vectors / np.clip(norms, 1e-12, None)

chunk_vectors = normalise(embed_texts([chunk["text"] for chunk in chunks]))
```

Retrieve the most relevant chunks:

```
def retrieve(question, top_k=5):
    question_vector = normalise(embed_texts([question]))[0]
    scores = chunk_vectors @ question_vector
    best = np.argsort(scores)[::-1][:top_k]
    return [{"chunks": chunks[index], "score": float(scores[index])} for index in best]
```

This example uses an in-memory NumPy matrix for learning. A larger project would normally use a persistent vector database with access control and metadata filtering.

6. Generate a grounded medical answer

RAG retrieves relevant document chunks before asking the language model to answer. The model is instructed to use only the supplied sources, cite them and admit when evidence is missing.

```
def answer_question(question, top_k=5):
    retrieved = retrieve(question, top_k=top_k)
    context = "\n\n".join(f"[Source {i+1}] {x['title']} | {x['publisher']} | {x['date']} | {x['url']}\n{x['text']}" for i, x in enumerate(retrieved))
    system_prompt = "You answer using only the supplied medical sources. If evidence is missing, say so. Do not diagnose, prescribe, or recommend."
    user_prompt = f"Question: {question}\n\nSources:\n{context}"
    response = client.chat.completions.create(model="YOUR_CHAT_MODEL", temperature=0, messages=[{"role": "system", "content": system_prompt}, {"role": "user", "content": user_prompt}])
    return response.choices[0].message.content, retrieved
```

Test the complete pipeline:

```
question = "What does the guideline say about prevention?"
answer, sources = answer_question(question)
print(answer)
for source in sources:
    print(source["title"], source["url"])
```

The answer should include source references and should state when the documents do not provide enough evidence.

7. How the complete project fits together

The extraction API and the RAG notebook are separate components. The API handles document ingestion and text extraction. The notebook handles chunking, embeddings, retrieval and question answering.

```
Document PDF
  | 1. POST /extract
  v
Extraction API
  | 2. JSON: text + page metadata
  v
Notebook / RAG pipeline
  | 3. chunks -> embeddings -> retrieval
  v
Language model
  | 4. grounded answer + citations
  v
User
```

Debug each boundary separately:

- First confirm that the API returns text.
- Then inspect the chunks and metadata.
- Then test retrieval with a known question.
- Only after retrieval works, connect the language model.
- Finally, test citations, refusal behaviour and medical-safety rules.

8. Learning checklist

Build the project in this order:

- Extract text from one PDF.
- Return the extracted text as JSON.
- Call the API from the notebook.
- Split the text into chunks.
- Create embeddings for the chunks.
- Retrieve relevant chunks for a question.
- Generate an answer using only the retrieved chunks.
- Add citations and medical-safety instructions.
- Test questions with known answers.

Before using a medical-document RAG prototype, check:

- Are the documents trusted and up to date?
- Does retrieval return the correct source section?
- Does the answer stay within the supplied sources?
- Are citations accurate?
- Does the system admit when evidence is missing?
- Does it avoid diagnosis and personalised treatment advice?
- Is private or identifiable patient information protected?

Summary

The key sequence is: understand the API request and response, build a small extraction API, call it from the notebook, then add retrieval and answer generation. Keeping each stage separate makes the project easier to test and understand.